CONCURRENCY AND COMPUTATION: PRACTICE AND EXPERIENCE
Concurrency Computat.: Pract. Exper. 2011; **23**:681–693
Published online 14 January 2011 in Wiley Online Library (wileyonlinelibrary.com). DOI: 10.1002/cpe.1686

Fast in-place, comparison-based sorting with CUDA: a study with bitonic sort

Hagen Peters*, †, Ole Schulz-Hildebrandt and Norbert Luttenberger

Research Group for Communication Systems, Department of Computer Science, Christian-Albrechts-University Kiel, Germany

SUMMARY

State-of-the-art graphics processors provide high processing power and furthermore, the high programmability of GPUs offered by frameworks like CUDA (Compute Unified Device Architecture) increases their usability as high-performance co-processors for general-purpose computing. Sorting is well investigated in Computer Science in general, but (because of this new field of application for GPUs) there is a demand for high-performance parallel sorting algorithms that fit with the characteristics of the modern GPU-architecture. We present a high-performance in-place implementation of Batcher's bitonic sorting networks for CUDA-enabled GPUs. Therefore, we assigned compare/exchange operations to threads in a way that decreases low-performance global-memory access and makes efficient use of high-performance shared memory. This greatly increases the performance of this in-place, comparison-based sorting algorithm. Our implementation outperforms all other algorithms in our tests when sorting 64-bit keys. It is the fastest comparison-based GPU sorting algorithm for 32-bit keys, being only outperformed by (non-comparison-based) radix sort when sorting sequences larger than 2^{23}. Copyright © 2011 John Wiley & Sons, Ltd.

Received 15 March 2010; Revised 30 June 2010; Accepted 18 September 2010

KEY WORDS: GPGPU; CUDA; parallel sorting; bitonic sort

INTRODUCTION

The processing power of modern desktop-GPUs has greatly increased in the recent years. In fact, it far exceeds the processing power (measured in FLOP/s) of desktop-CPUs. Since the release of frameworks like NVIDIA's CUDA (**C**ompute **U**nified **D**evice **A**rchitecture), the processing power has become GPUs is easily available for General-Purpose Computing on GPUs (GPGPU).

Many general-purpose applications require high-performance sorting algorithms, therefore many sorting algorithms have been explicitly developed for GPUs. Early implementations [1–5] were often based on bitonic sort [6]. Meanwhile many different sorting algorithms were developed for GPUs, many of them using CUDA. Harris *et al.* [7], Grand [8], and He *et al.* [9] implemented radix sort for GPUs. Sengupta *et al.* [10] were the first to implement quick sort, later Cederman and Tsigas [11] developed a more efficient implementation of quick sort. Sintorn *et al.* [12] presented a hybrid algorithm that combines merge sort and bucket sort. Satish *et al.* [13] developed new implementations of radix sort and merge sort for CUDA. Recently, Leischner *et al.* [14] introduced a sample sort implementation for CUDA. None of these implementations for GPU works in-place, which strongly reduces the maximum length of sequences that can be sorted.

*Correspondence to: Hagen Peters, Research Group for Communication Systems, Department of Computer Science, Christian-Albrechts-University Kiel, Germany.
†E-mail: hap@informatik.uni-kiel.de

Copyright © 2011 John Wiley & Sons, Ltd.

The early implementations of bitonic sort mentioned above were programmed using graphic APIs. The implementations focus on overcoming the limitations of these APIs including the lack of scattered writes, lack of shared memory, accessing main memory by textures only, etc. The performance of these implementations is not competitive (cf. [11, 14, 15]) with the newer sorting algorithms for GPUs. Although bitonic sort is an in-place algorithm these implementations are not in-place due to the limitations of the APIs and the underlying hardware. In this contribution we present an implementation of Batcher's bitonic sort for CUDA, which keeps the benefits of sorting networks, i.e. it works comparison-based and also in-place. In contrast to other implementations of bitonic sort, it is highly competitive with other sorting algorithms for GPUs, including non-comparison-based radix sort.

In the following section we give an introduction to bitonic sort and the next section briefly introduces CUDA. We present a basic implementation of bitonic sort for CUDA and our optimizations in *Implementation Design* section. In the *Evaluation* section we show and evaluate the performance measurements and comparisons to other GPU and CPU sorting algorithms. We summarize the results in the *Conclusion*.

INTRODUCTION TO BITONIC SORT

Bitonic Sort is a sorting network first devised by Ken Batcher [6]. A sorting network is a sorting algorithm where the sequence of comparisons (i.e. the order, direction, and number of comparisons) is data-independent and therefore the algorithm can be described through a static network of comparators, connected to each other with data lanes. Figure 2 presents a bitonic sorting network for an input sequence of length 8 and Figure 1 illustrates the divide-and-conquer design of bitonic sort presenting the recursive pseudocode.

The number of compare/exchange operations needed by bitonic sort for sorting a sequence of length n, and thereby the (sequential) complexity, is $O(n \cdot \log^2 n)$, which is not optimal. The strength of bitonic sort is that it is a sorting network, which can be implemented directly in hardware, but which is also adequate for generic parallel architectures since it works in-place, requires less inter-process communication and can be naturally implemented in SIMD architectures as NVIDIA's GPUs.

Bitonic merge

Bitonic sort is based on bitonic sequences, i.e. concatenations of two subsequences sorted in opposite directions. Given a bitonic sequence B with length $m = 2^r$, B can be sorted (i.e. merged) ascending (analog descending) in r steps. In the first step the pairs of elements

$$(\mathrm{B}[0], \mathrm{B}[m/2]), (\mathrm{B}[1], \mathrm{B}[m/2+1]), \ldots, (\mathrm{B}[m/2-1], \mathrm{B}[m-1])$$

are compared and exchanged if the first element is smaller than the second element. This results (as shown by Batcher [6]) in two bitonic subsequences:

$$(\mathrm{B}[0], \ldots, \mathrm{B}[m/2-1]) \quad \text{and} \quad (\mathrm{B}[m/2], \ldots, \mathrm{B}[m-1])$$

All elements in the first subsequence are smaller than any element in the second subsequence.

```
BSort(key* lo, int n, int dir) {
  if (n > 1) {
    m = n / 2;
    hi = lo + m;
    BSort(lo, m, ASC);
    BSort(hi, m, DSC);
    BMerge(lo, n, dir);
  }
}
```

```
BMerge(key* lo, int n, int dir) {
  if (n > 1) {
    m = n / 2;
    for (i = lo; i < lo + m; i++)
      compEx(i, i + m, dir);
    BMerge(lo, m, dir);
    BMerge(lo + m, m, dir);
  }
}
```

Figure 1. Pseudocode for bitonic sort.

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

In the second step the same procedure is applied to both subsequences, resulting in four bitonic subsequences. All elements in the first subsequence are smaller than any element in the second, all elements in the second subsequence are smaller than any element in the third and all elements in the third subsequence are smaller than any element in the fourth subsequence. The third, fourth,..., rth step are analog. Processing the rth step results in 2^r subsequences of length 1, all elements of a sequence being smaller than elements in following sequences. Thus sequence B is sorted.

Bitonic sort

Let A be an arbitrary input sequence to sort and let $n=2^k$ be the length of A. The process of sorting A then consists of k phases. The subsequences of length 2

$$(\mathrm{A}[0], \mathrm{A}[1]), (\mathrm{A}[2], \mathrm{A}[3]), \ldots, (\mathrm{A}[n-2], \mathrm{A}[n-1])$$

are bitonic sequences by definition. In the first phase these subsequences are sorted using bitonic merge (as shown above) alternating descending and ascending, which makes the subsequences of length 4 bitonic sequences:

$$(\mathrm{A}[0], \mathrm{A}[1], \mathrm{A}[2], \mathrm{A}[3]), \ldots, (\mathrm{A}[n-4], \mathrm{A}[n-3], \mathrm{A}[n-2], \mathrm{A}[n-1])$$

In the second phase these subsequences of length 4 are sorted alternating descending and ascending, resulting in subsequences of length 8 being bitonic sequences. In the rth phase of bitonic sort the total number of subsequences being sorted is 2^{k-r} and the length of each of these subsequences is 2^r. Sorting a sequence of length 2^r using bitonic merge consists of r steps. After the $(k-1)$th phase sequence A is a bitonic sequence. A is sorted in the last phase k.

Figure 2 shows the bitonic sorting network for input sequences of size $n=8$. The sorting network consists of $3=\log 8$ phases, phase p having p steps. Every step consists of $4=n/2$ compare/exchange operations.

Normalized bitonic sort

In our implementation we used a normalized variant of bitonic sort, i.e. the sorting direction of all compare/exchange operations is the same in all steps of all phases, while in the original bitonic sort in each phase adjacent sequences are sorted in opposite directions. In that case, the compare/exchange operations of the first step of each phase (i.e. step p of phase p) have to be

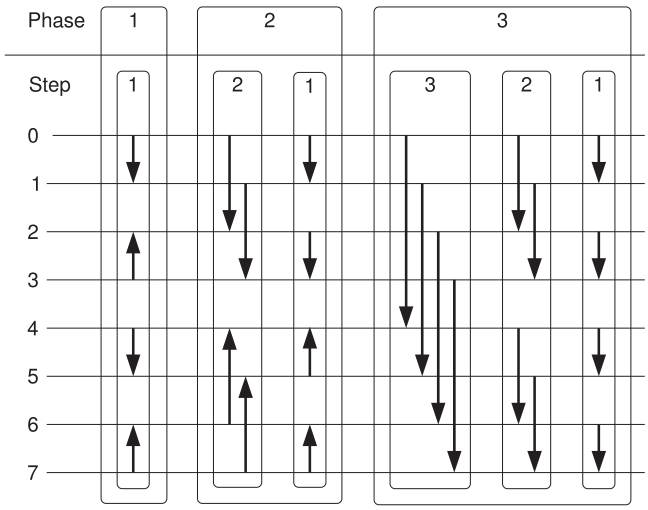


Figure 2. Bitonic sorting network.

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

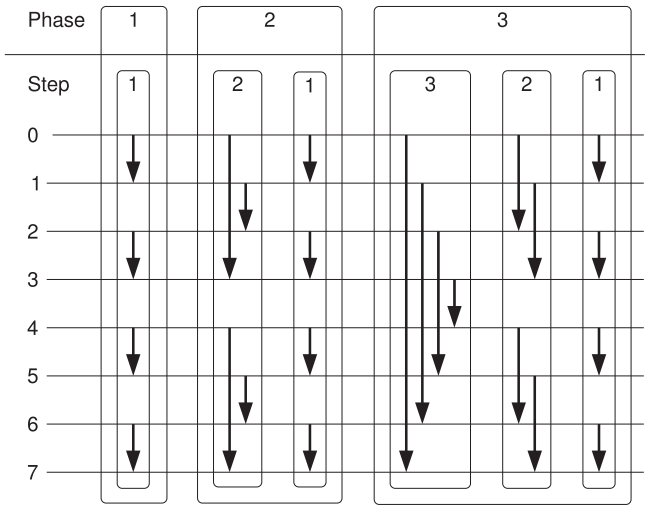


Figure 3. Normalized bitonic sorting network.

modified (indices of the second half of each input sequence are mirrored). Figure 3 presents the normalized bitonic sorting network for an input sequence of size $n = 8$.

Normalized bitonic sort is easier to implement in CUDA since every thread sorts its elements in the same direction. In addition, normalized bitonic sort can be easily adapted for input sequences of arbitrary length, while it is much more complex to process sequences of arbitrary length using original bitonic sort. On the other hand, normalized bitonic merge is less generic since each input sequence of phase p must consist of two sorted subsequences each of length 2^{p-1} (due to mirrored indices). In the original bitonic merge the input sequences consist of two sorted (in opposite direction) subsequences with total length 2^p. By this reason normalized bitonic merge is inadequate for merging sequences of different lengths, e.g. for external sorting [16].

CUDA

CUDA is a parallel programming model and software environment for NVIDIA's GPUs. CUDA allows the programmer to define functions, so-called kernels, which are executed on the GPU. One kernel is executed in parallel by a number of CUDA threads. Threads are organized in warps and blocks. A warp is a fixed-size group of threads (32 threads on GPUs with compute capability 1.x, 2.0 [17]), while a block consists of up to 16 (1.x) or 32 (2.0) warps. The number of warps per block and the number of blocks (and therefore the total number of threads) can be specified for each individual kernel launch. On up-to-date GPUs the maximum number of blocks is 2^{32}. To achieve a good utilization of all processing units on the GPU it is recommended to use more than one block per multiprocessor, where each block consists of multiple warps. Thus, when using a GTX280 GPU (30 multiprocessors) at least $2 \times 30 \times 64 = 7680$ threads have to be used.

A thread that executes a kernel has access to high-performance thread local registers with the same lifetime as the lifetime of the thread. Each thread block has a shared memory (visible to all threads of the block) with the lifetime of the block. The shared memory is as fast as registers, as long as there are no conflicting accesses by threads of the same block. All threads have random access to global memory that has the lifetime of the application, i.e. data reside in global memory for several kernel launches. In general, global memory is significantly slower than access to registers or shared memory. When threads of the same half-warp (upper or lower 16 threads of a warp) access the global memory simultaneously, the accesses can be coalesced into a single memory transaction, if the accessed memory lies in the same global memory segment. If not, the simultaneous access causes multiple sequential memory transfers.

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

CUDA provides a barrier mechanism for synchronization of threads of the same block, but does not provide any synchronization mechanisms for threads of different blocks. In general, the host is not blocked when launching a kernel, but it can explicitly wait for a kernel to finish (host synchronization). Since variables reside in global memory for multiple kernel launches, tasks can be synchronized by distributing the workload to multiple kernel launches using host synchronization, while keeping information in global memory.

Efficient synchronization and memory access are two of the most important factors that influence the performance of a GPGPU application. Since kernel launches cause small startup delay, the application should use CUDA's barrier mechanism instead of host synchronization or even better should avoid explicit synchronization. The host can only access the global memory of the GPU, thus global memory access cannot be completely avoided, but the global memory accesses of a kernel should be reduced to a minimum.

IMPLEMENTATION DESIGN

Basic approach

For a trivial implementation of bitonic sort with CUDA one could use an approach that stepwise follows the implementation of bitonic sort by sorting networks: Bitonic sort could then be implemented by a sequence of kernel launches, each launch processing all compare/exchange operations of one step of one phase. Every kernel launch would consist of 2^{k-1} threads, each thread processing one compare/exchange operation. Figure 4 presents four threads, each loading two elements (lanes 0,4, 1,5, 2,6, 3,7) from global memory and processing one compare/exchange operation of step 3 in phase 3.

This implementation is optimal in the sense that it provides a good utilization of the GPU, since many threads can process in parallel, and memory accesses and workload are uniformly distributed to threads. The main disadvantage is the excessive use of global memory which significantly increases the runtime of the algorithm. Since all operations of one step are processed in an individual kernel launch, each compare/exchange operation causes at least two read accesses to global memory.

Sorting a sequence of length $n=2^k$ requires k phases. The rth phase consists of r steps. Each step consists of $n/2$ compare/exchange operations, thus in the trivial implementation each step causes $2\cdot 2^{k-1}=2^k=n$ read accesses to global memory. Thus the total number of read accesses

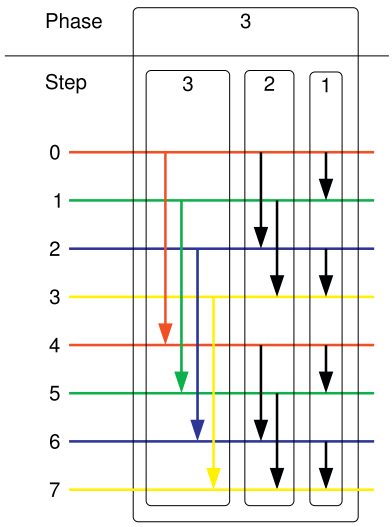


Figure 4. Partition example 1.

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

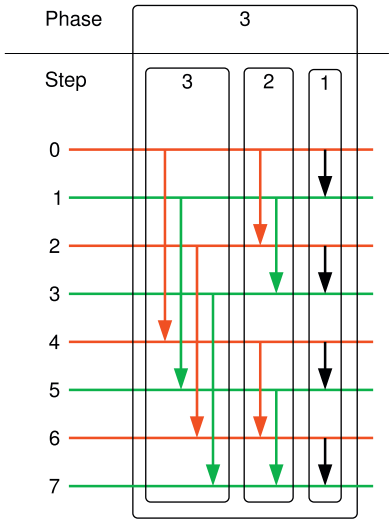


Figure 5. Partition example 2.

in this case is

$$n \cdot \sum_{r=1}^{k} r = n \cdot \frac{k \cdot (1+k)}{2} = n \cdot \frac{(\log n) \cdot (1+\log n)}{2} \tag{1}$$

The number of write accesses is at most the same.

Reducing global memory access

Our approach to significantly reduce the runtime of the algorithm is to reduce the number of accesses to global memory and the number of kernel launches. Therefore we carefully partition the set of operations (set of data elements, respectively) into disjunct subsets that can be processed by single threads, such that operations of multiple consecutive steps can be processed by one thread in one kernel launch without synchronization to other threads and each element has to be read from or written to global memory at most once.

Figure 5 shows an example of such a partition in step 3 of phase 3. The partition consists of two subsets, one marked by dashed lines and arrows, the other marked by dotted lines and arrows. Each subset consists of four data elements that affect exactly two compare/exchange operations in both steps 3 and 2. Thus, a thread that processes both operations of one subset in step 3 can also process both operations in step 2 nonstop, because both operations in step 2 were only affected by operations of step 3 that belong to the same subset. While the thread processes four operations of two steps the elements reside in thread-local registers, thus the elements are read and written from and to global memory only once for processing two steps (and not twice like in the basic implementation, Figure 4).

The basic partition consists of four subsets (each consists of two data elements), thus up to four threads can process step 3 in parallel. In contrast, the partition in Figure 5 consists of only two subsets, thus in this case only two threads can process step 3.

In the following we give a formal definition of the partitions used in our algorithm. Consider a sequence A of size 2^k to be the input sequence to sort. The set of indices in A is $I = \{0, \ldots, 2^k - 1\}$. We define $COEX_{p,s}$ to be the set of compare/exchange operations that belong to step s of phase p of bitonic sort. The commutative representation of one compare/exchange operation in $COEX_{p,s}$ is $coex_{p,s}(a, b) = coex_{p,s}(b, a)$. The whole set of compare/exchange operations processed by bitonic sort for sorting A is

$$COEX = \bigcup_{p=1}^{k} \left(\bigcup_{s=1}^{p} COEX_{p,s} \right)$$

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

We define an operator K that maps sets of compare/exchange operations to the sets of indices of the sequence that are affected by these operations. Furthermore, we define a class of operators $N_{p,s}$ that map a set of sequence positions J to the set of compare/exchange operations that affect array positions in J in step s of phase p:

$$K : \mathscr{P}(COEX) \to \mathscr{P}(I);\ M \mapsto \{i \mid coex_{\bullet,\bullet}(A[i], \bullet) \in M\}$$

$$N_{p,s} : \mathscr{P}(I) \to \mathscr{P}(COEX_{p,s});\ J \mapsto \{coex_{p,s}(A[i], \bullet) \in COEX_{p,s} \mid i \in J\}$$

In a single step s of a phase p any element of A is involved in exactly one compare/exchange operation ($\forall i \in I : |N_{p,s}(\{i\})| = 1$). There are no dependencies between operations in the same step of the same phase, thus all operations of $COEX_{p,s}$ can be done (asynchronously) in parallel, as it is done in the basic implementation.

To ensure correctness, bitonic sort requires that before an operation $c \in COEX_{p,s}$ is processed, both corresponding operations in the preceding step, more precisely, the operations in $N_{p,s+1}(K(\{c\}))$ must be already processed—the same requirement holds for elements in those sets, i.e. the four corresponding operations in step $s+2$ must be already processed and so on. Again have a look at Figure 5. In step 2 each red colored operation corresponds to both red colored operations in step 3.

A general definition of a partition P of I is

$$P = \{I_1, \ldots, I_m\} \quad \text{is partition of } I \iff \bigcup_{i=1}^{m} I_i = I \forall i, j \in \{1, \ldots, m\}, i \neq j : I_i \cap I_j = \emptyset$$

Let P be a partition of I. P ensures a uniform distribution of memory access and workload to threads, if P holds

$$\forall J \in P : |J| = \frac{n}{m}, |N_{p,s}(J)| = \frac{1}{2} \cdot |J| \tag{2}$$

In fact, if P holds (2) then P is a partition of equal-sized sets of indices that induce sets of the half size of compare/exchange operations, thus

$$\{N_{p,s}(J) \mid J \in P\} \quad \text{is a partition of } COEX_{p,s} \tag{3}$$

In simple words, all operations in $COEX_{p,s}$ that affect elements in $J \in P$ do not affect elements that are not in J. Since the size of induced sets of operations is half the size of sets of indices, all operations in $COEX_{p,s}$ are processed.

We define the degree of P in step s of phase p to be the maximum number of successive steps for which it holds (2) and (3). More precisely, the degree d of P in step s of phase p is the maximum number z for which it holds

$$\forall a \in \{1, \ldots, z\} : \{N_{p,s-a+1}(J) \mid J \in P\} \quad \text{is a partition of } COEX_{p,s-a+1} \tag{4}$$

In the example in Figure 5 the used partition in step 3 of phase 3 is $\{\{0, 2, 4, 6\}, \{1, 3, 5, 7\}\}$. This partition of indices induces a partition of operations in step 3 of phase 3 and in step 2 of phase 3 (dashed, dotted arrows) but not in step 1—thus the degree of this partition is 2.

The basic implementation (Figure 4) of bitonic sort uses partitions of degree 1. If we use a partition of degree $d \geq 2$ in step s of phase p, we could process compare/exchange operations of d steps in a single kernel launch. More precisely, using a partition P that holds (2), all threads have to access $n/|P|$ elements in memory and can then process compare/exchange operations (in the correct order) of d steps nonstop, without synchronization to other threads. All operations of d consecutive steps can be done a in single kernel launch without increasing the number of memory accesses per kernel launch. In this case, the overall reading memory access is divided by the degree d of P compared to that in the basic implementation and also the number of kernel launches is divided by d. This significantly reduces the runtime of the algorithm.

The general definition of a partition in step s of phase p with degree $d \leq s$ is

$$P^d_{p,s} := \bigcup_{i \in I} K(N_{p,s}(K(N_{p,s-1}(\ldots K(N_{p,s-(d-1)}(\{i\}))\ldots)))) \tag{5}$$

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

In the following a kernel launch that processes a partition of degree d is called d-multistep and one element of a partition (the indices and the induced operations that have to be processed by a single thread) is called job or d-job, respectively.

Constructing a partition

A partition of degree d in step s of phase p can be defined through the definition of the partition of one of the subsequences of length 2^s that are processed in step s of phase p. The partitions of the other subsequences are analog.

In step s of phase p the subsequences are of length 2^p. The eth job in the partition is then defined through

$$\bigcup_{i=0}^{2^{d-1}-1} \{e+i\cdot 2^{p-d}, e+i\cdot 2^{p-d}+2^{p-1}\} \tag{6}$$

For example, the job e of the 1-multistep in step 3 in Figure 4 is defined by

$$\bigcup_{i=0}^{2^0-1} \{e+i\cdot 2^2, e+i\cdot 2^2+2^2\} = \{e, e+4\}$$

Thus, the trivial partition consists of indices of a single compare/exchange operation. In contrast, in Figure 5 the jobs of the depicted 2-multistep are defined by

$$\bigcup_{i=0}^{2^1-1} \{e+i\cdot 2^1, e+i\cdot 2^1+2^2\} = \{e, e+4\}\cup\{e+2, e+6\}$$

Obviously, the size of a d-job, i.e. the number of indices and by that the number of induced operations, depends on d. Using formula 6 it follows that the number of indices that belong to one d-job is 2^d, the number of compare/exchange operations is 2^{d-1}.

In our implementation of bitonic sort we use partitions of at most degree 4, since the number of registers (which hold the elements) is bounded. Additionally, an increasing degree results in an increasing workload per thread, which decreases the total number of threads and blocks (in the above example the first partition consists of four, the second of four jobs). Thus a higher degree may result in under-utilization of the GPU.

Using shared memory

In the above sections we described which compare/exchange operations (which jobs, respectively) should be done by a single thread, so that the elements to process reside in thread-local registers and the number of global memory accesses is reduced. Of course it is possible to adapt this technique from threads to blocks of threads: A single job is processed by one block instead of one thread, storing its elements in shared memory instead of registers and distributing the compare/exchange operations of the job among the threads.

The amount of shared memory available to one block is much greater than the amount of register memory of one thread, thus we can use partitions of higher degree. Unfortunately, the elements that belong to one job do not lie consecutively in global memory in general (Figure 5), i.e. accessed memory does not lie in the same memory segment. For this reason it is not possible to coalesce these memory transactions, which makes this approach unprofitable.

There is another way to benefit from the higher performance of shared memory compared with global memory: if a subsequence of length 2^s processed in step s completely fits into shared memory of one block, the block first transfers the subsequence into shared memory. Later the threads of the block process the steps $s, s-1, \ldots, 1$ using the approach described above, but accessing shared instead of global memory and using efficient block-synchronization mechanisms between multisteps instead of host-synchronization.

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

In our tests we used a GTX280 which provides 16 KByte of shared memory per multiprocessor, thus it is possible to store a (32 bit)-sequence of length $4K(2^{12})$ in shared memory. Hence sequences processed in step $s \leq 12$ completely fit into shared memory and can be sorted there without further accesses to global memory. Furthermore, since within the first p phases no sequences of length larger than 2^p are processed, it is possible to completely process the first $p \leq 12$ phases within the shared memory.

Since the number of steps that can be processed in shared memory is memory bounded, only a constant number of phases (the first ones) and a constant number of steps (the last ones) in each phase can be efficiently processed using shared memory. This number is independent of the length of the sequence to sort. For longer sequences the number of phases, and by that the number of steps per phase, increases, but since the number of steps processed in shared memory is constant, the positive impact decreases.

In our implementation we process, depending on the key-type of the sequence to sort and if it is key-value sorting, up to $s = 9$ phases completely in the shared memory and in each phase greater than s, the last s steps are also processed in the shared memory.

Summary

The use of partitions with degree $d > 1$ and the use of the shared memory greatly decreases the number of global memory accesses. The number of read accesses for sorting a sequence of length n in the basic implementation is $n \cdot ((\log n) \cdot (1 + \log n))/2$. When processing the first nine phases and the last nine steps of greater phases in the shared memory and using 4-multisteps the number of global memory accesses is

$$n \cdot \left(1 + \sum_{r=10}^{\log n} \left(\left\lceil \frac{r-9}{4} \right\rceil + 1 \right)\right) \tag{7}$$

This formula also demonstrates the lesser impact of the use of shared memory for larger sequences: the use of shared memory for up to nine steps (independent of sequence length) causes the sum to start at $r = 10$ and decreases the inner expression by 9, whereas the use of 4-multisteps causes the inner expression to be divided by 4. Obviously, the impact decreases if n goes towards larger numbers.

For example, when sorting a sequence of length 2^{20}, the basic implementation transfers the total sequence 210 times from global memory into registers, whereas the optimized implementation only transfers the sequence 33 times into registers.

It is obvious that the threads of a block can transfer a sequence of consecutive elements into shared memory and back solely using coalesced memory transfers. The memory access in larger phases is coalescent too, thus all global memory transfers are coalescent.

Since a constant number of phases and a constant number of steps of greater phases is processed in shared memory, the use of shared memory is less advantageous for larger sequences than for smaller sequences.

IMPLEMENTATION

Using our optimizations, our implementation of bitonic sort is straightforward. Pseudocode is given in Figure 6. It consists of two parts: in the first part the first `firstPh` phases are processed within one single kernel launch (`BS_firstPh`) using shared memory. In the second part the remaining phases are processed. In each of these phases at the beginning the steps are calculated using 4-multisteps (`BS_4multistep`), 3-multiSteps (`BS_3multistep`), 2-multisteps (`BS_2multistep`), and 1-multisteps (`BS_1multistep`). The last `lastSt` steps of each phase are again processed within one single kernel launch (`BS_lastSt`) using shared memory.

The partitioning of compare/exchange operations is deterministic and only depends of the type of the partition (1,2,3,4-job), the phase and the step. Thus, accordant to its thread-identifier a thread simply computes within a kernel what data elements it has to fetch from global memory and

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

```
BS(key* lo, int n, int dir, int phases, int firstPh, int lastSt) {

    BS_firstPh<<<grid, block, 0, 0>>>(lo, firstPh, n, dir);

    for (int ph = firstPh + 1; ph <= phases; ph++) {
        int st = ph;

        for (; st >= lastSt + 4; st -= 4) {
            BS_4multistep<<<grid,block, 0, 0>>>(lo, ph, st, dir);

        for (; st >= lastSt + 3; st -= 3) {
            BS_3multistep<<<grid,block, 0, 0>>>(lo, ph, st, dir);

        for (; st >= lastSt + 2; st -= 2) {
            BS_2multistep<<<grid,block, 0, 0>>>(lo, ph, st, dir);

        for (; st >= lastSt + 1; st -= 1) {
            BS_1multistep<<<grid,block, 0, 0>>>(lo, ph, st, dir);

        BS_lastSt<<<grid,block, 0, 0>>>(lo, ph, lastSt, dir);
    }
}
```

Figure 6. Pseudocode for bitonic sort with CUDA.

what compare/exchange operations it has to process (according to the above-mentioned partition construction). Thus, there is no kind of control-thread needed.

EVALUATION

Sorting is a fundamental problem in computer science and is a part of many applications for GPUs (e.g. MapReduce [18, 19], databases [20, 21]). Because of the multiple uses of sorting algorithms, datasets of all sizes are sorted, thus our test includes sequences ranging from 2^{10} to the maximum length fitting into GPU memory. The type of the data processed by the mentioned applications differs. Especially the importance of 64-bit integer and 64-bit floating point data for GPGPU is growing, e.g. NVidia's *Fermi* architecture is specifically designed for double precision [22]. For this reason we tested our implementation using randomly generated sequences of 32-bit integers, 32-bit floats, 64-bit integers, and (32-bit integer, 32-bit integer) key-value pairs.

Since our bitonic sort is based on sorting networks, its performance is nearly independent of key distribution. Owing to this, on the one hand the runtime of our implementation is precisely predictable, on the other hand it cannot benefit from special key distributions (cf. sample sort [14]) or key entropy (radix sort, merge sort [13]). Nevertheless, in most cases the influence of key distribution and entropy on performance is marginal for all tested algorithms, thus we tested uniformly distributed sequences only.

All tests were done on an ASUS Z8PE-D18 equipped with two Quad-Core Xeon processors and a GTX280 GPU.

GPU sorting

We compared our optimized bitonic sort to the best-performing algorithms for GPUs: the quick sort implementation from Cederman and Tsigas [11], the radix sort implementation in the CUDPP library [7], the radix sort and the merge sort algorithm from Satish *et al.* [13] from the Thrust library and to the sample sort algorithm from Leischner *et al.* [14]. To show the impact of our optimization to the runtime of our implementation we also tested the basic implementation and an implementation that uses shared memory for the first phases and the last steps of greater phases, but only uses partitions of degree 1 (1-multistep) ('Bitonic Sort, semi' in the figure).

Among the tested algorithms bitonic sort is the only one that works in-place and the other algorithms need more or less additional memory, therefore the maximum length of a sequence that

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

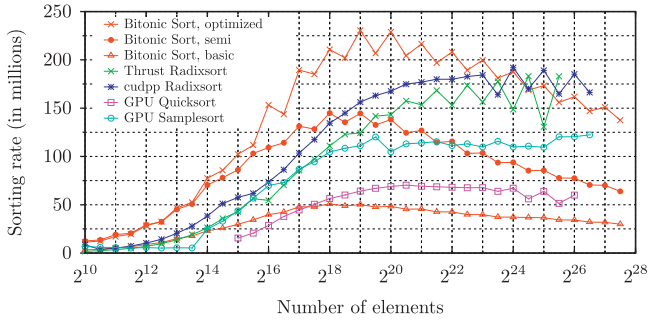


Figure 7. Sorting 32-bit integers.

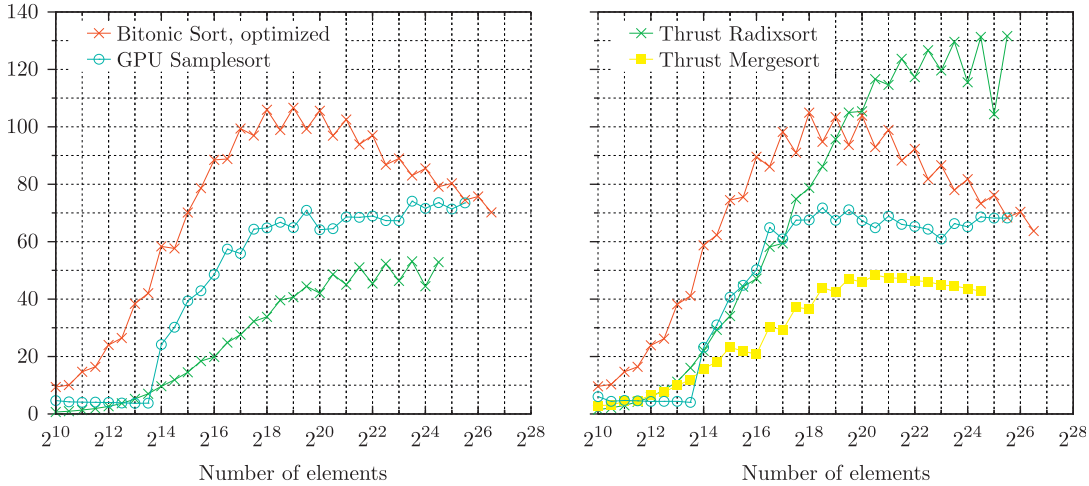


Figure 8. Sorting rates for 64-bit long (left) and (32-bit, 32-bit) key-value pairs (right).

can be sorted differs for each algorithm. Furthermore, not all the implementations were able to sort all data types, hence we could not include every implementation in every test.

For comparing the GPU algorithms, we measured the GPU runtime only, thus the time needed to copy the sequences from CPU memory to GPU memory and back is not included. All figures show the sorting rate, i.e. the number of elements divided by the runtime, on the y-axis and on the length of the input sequence at the x-axis.

Figure 7 presents the sorting rates for 32-bit integers (sorting rates for 32-bit floats are roughly the same). Our optimized bitonic sort is faster than the other comparison-based sorting algorithms, GPU quick sort and GPU sample sort, for all sequence lengths and it is faster than all other sorting algorithms for sequences up to a length of 2^{23}. For some larger sequences it is outperformed by both radix sorts, but still performs competitively.

Compared with other algorithms, the sorting rate of bitonic sort strongly depends on the length of the input sequence. On the one hand this is due to the fact that the algorithm cost w.r.t. the sequence length n is $O(n \cdot \log^2 n)$ for bitonic sort, whereas $O(n \cdot \log n)$ for merge sort, quick sort and sample sort and only $O(n)$ for radix sort. Thus, the sorting rates should converge to $O(1/\log^2 n)$, $O(1/\log n)$, and $O(1)$, respectively. On the other hand our implementation benefits less from the use of shared memory in larger phases (cf. Section *Using Shared Memory*).

In Figure 8 the results for sorting 64-bit integers and for sorting key-value pairs of 32-bit integers are shown. The results of the comparison-based algorithms quick sort, sample sort, and bitonic sort are nearly the same in both cases. Owing to the fact that the cost of radix sort depends on key size, the performance of radix sort is significantly lower when sorting 64-bit keys. Thus, bitonic sort

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

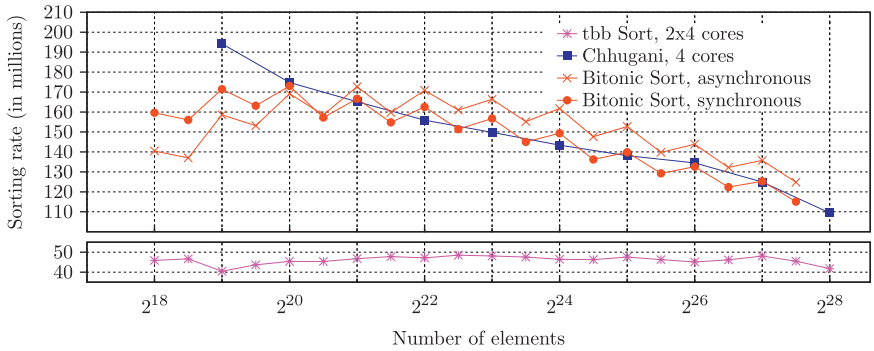


Figure 9. Sorting 32-bit integer including memory transfer.

is the fastest algorithm when sorting 64-bit keys but is outperformed by radix sort when sorting sequences of key-value pairs larger than 2^{19}.

CPU sorting

In general, we suppose sorting to be part of a larger computation that is processed on the GPU. However, it is also an interesting question whether it is beneficial to use the GPU for sorting only, i.e. sorting is the only computation performed on the GPU. In this scenario the time for data transfers to and from the GPU has to be considered.

Figure 9 presents the sorting rates for sorting 32-bit integers for the `tbb::parallel_sort` algorithm from Intel's TBB-Library [23] and the sorting algorithm for Intel CPUs developed by Chhugani *et al.* [24] and the sorting rates of our bitonic sort including memory transfer.

While the results given for `tbb::parallel_sort` are measured in our tests, the results for the implementation of Chhugani *et al.* are taken from the original article [24]. In this paper a single Intel Q9550 Quad-Core CPU with a clock speed of 3.22 MHz is used. Since their implementation bases on the use of cache memory, its performance when using two processors (as on our test system) is unclear.

The figure presents the results for a 'synchronous' and an 'asynchronous' version of bitonic sort. The synchronous version sequentially transfers the sequence to sort to the GPU memory, sorts the sequence and transfers it back to the CPU memory. The asynchronous version makes use of the capability of *asynchronous memory transfer* of some current NVidia GPUs, i.e. it overlaps parts of memory transfer with GPU computation (similar to the approach in [16]).

The implementation from Chhugani *et al.* and both versions of bitonic sort outperform `tbb::parallel_sort` for tested sequence lengths. The implementation from Chhugani *et al.* performs better on smaller sequences while both versions of bitonic sort are hampered by the small constant delays caused by memory transfers and kernel launches. Thus, both versions of bitonic sort are outperformed for sequences up to a length of 2^{21}. The asynchronous version of bitonic sort uses multiple delay producing memory transfers and kernel launches and by this cannot benefit from asynchronous memory transfer for these sequences. For sequences larger than 2^{21} elements the synchronous version of bitonic sort performs competitively with the implementation from Chhugani *et al.* while the asynchronous version is even faster.

CONCLUSION

We have presented an efficient implementation of bitonic sort for NVIDIA's GPUs. To achieve this we carefully optimized our implementation with respect to the number of accesses to global memory and the number of kernel launches. It is the first efficient in-place implementation of a sorting algorithm for CUDA we found in the literature and it is also comparison-based. Although bitonic sort suffers from cost $O(n \cdot \log^2 n)$, the results of our tests show that it is highly competitive

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe

with all other GPU sorting algorithms except for key-value sorting, where it is outperformed by radix sort only.

Our bitonic sort is the fastest comparison-based algorithm known to the authors and even outperforms non-comparison-based algorithms when sorting 64-bit keys.

REFERENCES

1. Kapasi UJ, Dally WJ, Rixner S, Mattson PR, Owens JD, Khailany B. Efficient conditional operations for data-parallel architectures. *MICRO 33*: *Proceedings of the 33rd Annual ACM/IEEE International Symposium on Microarchitecture*. ACM: New York, NY, U.S.A., 2000; 159–170.
2. Purcell TJ, Donner C, Cammarano M, Wann Jensen H, Hanrahan P. Photon mapping on programmable graphics hardware. *HWWS '03*: *Proceedings of the ACM SIGGRAPH/EUROGRAPHICS Conference on Graphics Hardware*. Eurographics Association, Aire-la-Ville, Switzerland, 2003; 41–50.
3. Kipfer P, Segal M, Westermann R. Uberflow: A GPU-based particle engine. *HWWS '04*: *Proceedings of the ACM SIGGRAPH/EUROGRAPHICS Conference on Graphics Hardware*. ACM: New York, NY, U.S.A., 2004; 115–122.
4. Govindaraju N, Raghuvanshi N, Henson M, Manocha D. A cache-efficient sorting algorithm for database and data mining computations using graphics processors. *Technical Report*, University of North Carolina-Chapel Hill, 2005.
5. Greb A, Zachmann G. Gpu-abisort: Optimal parallel sorting on stream architectures. *20th International Parallel and Distributed Processing Symposium* (*IPDPS 2006*), Rhodes Island, Greece, 2006.
6. Batcher KE. Sorting networks and their applications. *AFIPS Spring Joint Computer Conference*, Atlantic City, U.S.A., 1967.
7. Harris M, Sengupta S, Owens JD. Parallel prefix sum (scan) with cuda. *GPU Gems 3*. Addison-Wesley: Reading, MA, 2007.
8. Grand SL. Broad-phase collision detection with cuda. *GPU Gems 3*. Addison-Wesley: Reading, MA, 2007.
9. He B, Govindaraju NK, Luo Q, Smith B. Efficient gather and scatter operations on graphics processors. *SC '07*: *Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*. ACM: New York, NY, U.S.A., 2007; 1–12.
10. Sengupta S, Harris M, Zhang Y, Owens JD. Scan primitives for GPU computing. *GH '07*: *Proceedings of the 22nd ACM SIGGRAPH/EUROGRAPHICS Symposium on Graphics Hardware*. Eurographics Association, Aire-la-Ville, Switzerland, 2007; 97–106.
11. Cederman D, Tsigas P. A practical quicksort algorithm for graphics processors. *ESA '08*: *Proceedings of the 16th Annual European Symposium on Algorithms*. Springer: Berlin, Heidelberg, 2008; 246–258.
12. Sintorn E, Assarsson U. *Fast Parallel GPU-sorting using a Hybrid Algorithm*, vol. 68. Academic Press, Inc.: Orlando, FL, U.S.A., 2008; 1381–1388.
13. Satish N, Harris M, Garland M. Designing efficient sorting algorithms for many core GPUs. *Proceedings 23rd IEEE International Parallel and Distributed Processing Symposium*, Rome, Italy, May 2009.
14. Leischner N, Osipov V, Sanders P. GPU sample sort. *CoRR*; abs/0909.5649, 2009.
15. Peters H, Schulz-Hildebrandt O, Luttenberger N. Fast in-place sorting with CUDA based on bitonic sort. *PPAM09*: *Proceedings of the International Conference on Parallel Processing and Applied Mathematics*, Wroclaw, Poland, September 2009.
16. Peters H, Schulz-Hildebrandt O, Luttenberger N. Parallel external sorting for CUDA-enabled GPUs with load balancing and low transfer overhead. *IPDPS2010*: *Proceedings of the 24th IEEE International Parallel & Distributed Processing Symposium*, *Workshops and Phd Forum*, Atlanta, U.S.A., April 2010.
17. NVIDIA. Nvidia CUDA programming guide. Available at: http://developer.nvidia.com/object/gpucomputing.html [December 2010].
18. He B, Fang W, Luo Q, Govindaraju NK, Wang T. Mars: A mapreduce framework on graphics processors. *PACT '08*: *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*. ACM: New York, NY, U.S.A., 2008; 260–269.
19. Dean J, Ghemawat S. Google Inc. Mapreduce: Simplified data processing on large clusters. *OSDI'04*: *Proceedings of the Sixth Conference on Symposium on Opearting Systems Design and Implementation*, New York, U.S.A., USENIX Association, 2004.
20. Bakkum P, Skadron K. Accelerating SQL database operations on a GPU with CUDA. *GPGPU'10*: *Proceedings of the Third Workshop on General-Purpose Computation on Graphics Processing Units*. ACM: New York, NY, U.S.A., 2010; 94–103.
21. Govindaraju, Naga, Gray, Jim, Kumar, Ritesh, Manocha, Dinesh, Gputerasort: High performance graphics co-processor sorting for large database management. *SIGMOD'06*: *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data*. ACM: New York, NY, U.S.A., 2006; 325–336.
22. NVIDIA. Fermi compute architecture white paper. Available at: http://www.nvidia.com/object/fermi_architecture.html [December 2010].
23. http://www.threadingbuildingblocks.org/ [April 2000].
24. Chhugani J, Nguyen AD, Lee VW, Macy W, Hagog M, Chen Y-K, Baransi A, Kumar S, Dubey P. Efficient implementation of sorting on multi-core SIMD CPU architecture. *Proceedings of the VLDB Endowment* 2008; **1**(2):1313–1324.

Copyright © 2011 John Wiley & Sons, Ltd.

DOI: 10.1002/cpe